

# Reviewer Pack — Persistent Homology Barcode Length Bounds Tseitin Resolution...

Entry d6d8ca13e5b3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 18:18:27 UTC

## Persistent Homology Barcode Length Bounds Tseitin Resolution Length

**Entry ID:** d6d8ca13e5b3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 18:18:27 UTC

### 1. Conjecture

**Field A** (mathematical branch): Persistent Homology **Field B** (complexity object): Resolution Length of Tseitin Formulas

#### Statement:

For a Tseitin formula on graph  $G$ , the resolution length is  $\geq 2^{\Omega(v(G))}$ , where  $v(G)$  is the maximum length of a persistent homology barcode in the clique complex of  $G$ . If  $G$  is not an expander,  $v(G) = O(1)$ .

#### Rationale (proposer's reasoning):

Persistent homology captures topological complexity of clique complexes, which may correlate with the structural entanglement in Tseitin formulas. Expander graphs' high connectivity forces long barcodes, suggesting a direct link to resolution hardness.

**Taxonomy category:** TSEITIN\_RES (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 812a888ab622b8a9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |

| Barrier     | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|-------------|---------|------------|------------------|------------------|
| KARP_LIPTON | SAFE    | 0.95       | SAFE             | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            for j in range(n):
                A[i][j] /= pivot
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0]*p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def dpll(clauses, assignment):
        if not clauses:
            return True
        unit = None
        for clause in clauses:
            if len(clause) == 1:
```

```

        unit = clause[0]
        break
    if unit is not None:
        new_assignment = assignment.copy()
        new_assignment[unit] = True
        if dpll([c for c in clauses if unit not in c and -unit not in c], new_assignment):
            return True
        new_assignment[unit] = False
        if dpll([c for c in clauses if unit not in c and -unit not in c], new_assignment):
            return True
    else:
        literal = random.choice(clauses[0])
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        if dpll(clauses, new_assignment):
            return True
        new_assignment[literal] = False
        if dpll(clauses, new_assignment):
            return True
    return False

def tseitin_formula(n):
    clauses = []
    for i in range(1, n+1):
        clauses.append([i])
    for i in range(1, n+1):
        for j in range(i+1, n+1):
            clauses.append([-i, -j, -(n+i+j)])
            clauses.append([i, j, n+i+j])
    return clauses

def clique_complex(n):
    G = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if random.choice([True, False]):
                G[i][j] = 1
                G[j][i] = 1
    return G

def persistent_homology(G):
    n = len(G)
    simplices = []
    for i in range(n):
        simplices.append([i])
    for k in range(2, n+1):
        new_simps = []
        for s in simplices:
            for j in range(k-1):
                if s[j] not in G[s[-1]]:
                    break
            else:
                new_simps.append(s + [k-1])
        simplices.extend(new_simps)
    barcode = {}
    for s in simplices:

```

```

        birth = math.inf
        death = -math.inf
        for i in range(len(s)-1):
            if G[s[i]][s[i+1]]:
                birth = min(birth, max([G[s[j]][s[k]] for j in range(i) for k in range(i+2, len(s))]))
                death = max(death, min([G[s[j]][s[k]] for j in range(i+1) for k in range(i+2, len(s))]))
            if birth < math.inf and death > -math.inf:
                barcode[birth] = death
        return barcode

def resolution_length(clauses):
    assignment = {}
    return dpll(clauses, assignment)

n = random.randint(5, 40)
G = clique_complex(n)
barcode = persistent_homology(G)
nu_G = max(barcode.values()) if barcode else 1
clauses = tseitin_formula(n)
resolution_len = resolution_length(clauses)

return {
    "metric_name": "resolution_length",
    "metric_value": resolution_len,
    "instances_tested": 1,
    "conjecture_holds": resolution_len >= 2**(0.2 * nu_G),
    "counterexample": "" if resolution_len >= 2**(0.2 * nu_G) else f"Graph with n={n}, barcode={barcode}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Graph with n={results[0]['instances_tested']}, seed={first_failing_seed}\"")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_support_fraction support_fraction={support_fraction}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_411c0210.py", line 145, in <module>
    result = run_trial(seed)
             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_411c0210.py", line 126, in run_trial
    barcode = persistent_homology(G)
             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_411c0210.py", line 103, in persistent_h
    if s[j] not in G[s[-1]]:
       ~^^
IndexError: list index out of range
```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed with IndexError before producing data, preventing evaluation of conjecture |  
next: Debug persistent\_homology function's indexing logic, particularly the line checking  
s[j] in G[s[-1]]

## 11. Audit log (LLM calls)

**Total LLM calls:** 13

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 103581       |
| 2  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 138322       |
| 3  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 55156        |
| 4  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 138176       |
| 5  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 96589        |
| 6  | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 32115        |
| 7  | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 27306        |
| 8  | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 20525        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 23047        |
| 10 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16676        |
| 11 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15484        |
| 12 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15727        |
| 13 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 24176        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 706882 ms total latency. Provider mix:  
{'ollama\_remote': 13}

(full prompt+response transcripts available in research/audit/d6d8ca13e5b3.jsonl)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d6d8ca13e5b3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/d6d8ca13e5b3.tar.gz` (if generated)